

REST API - CRUD Operations using ResponseEntity

#SKILL - 7 → REST API - CRUD Operations using ResponseEntity

Prerequisites:

- General Idea on Spring Boot MVC Architecture
- Knowledge of Service and Controller layers

A university application needs backend services to manage **course details** such as **courseId**, **title**, **duration**, and **fee**. The admin should be able to **add**, **update**, **delete**, and **view** course information. The system must return proper **HTTP status codes** and **structured responses** for reliable communication between client and server.

Your task is to implement **REST endpoints** using **ResponseEntity** to perform complete CRUD operations on course data.

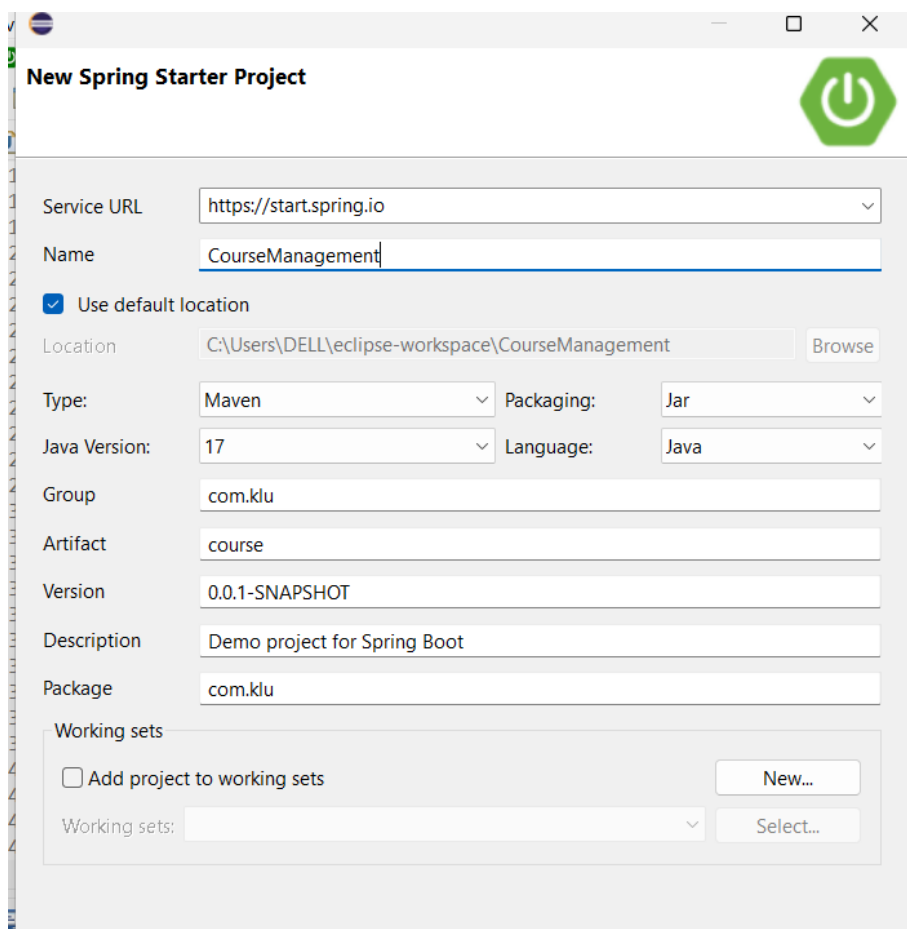
Tasks:

1. Create a **Course** entity with fields: **courseId**, **title**, **duration**, **fee**.
2. Create a **Service** layer that performs full CRUD operations.
3. Implement REST endpoints using **@PostMapping**, **@PutMapping**, **@DeleteMapping**, and **@GetMapping**.
4. Use **ResponseEntity** to return:
 - a. **Success messages**
 - b. **Error messages**
 - c. **Status codes** such as **OK**, **CREATED**, **NOT_FOUND**, **BAD_REQUEST**
5. Add a search endpoint **/courses/search/{title}** to filter courses by title.
6. Test all endpoints (valid and invalid cases) using **Postman**.
7. **Push the Spring Boot project to GitHub**.

1. Open Eclipse IDE

2. Go to File → New → Spring Starter Project

Fill details:



New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:

2. Add Dependencies

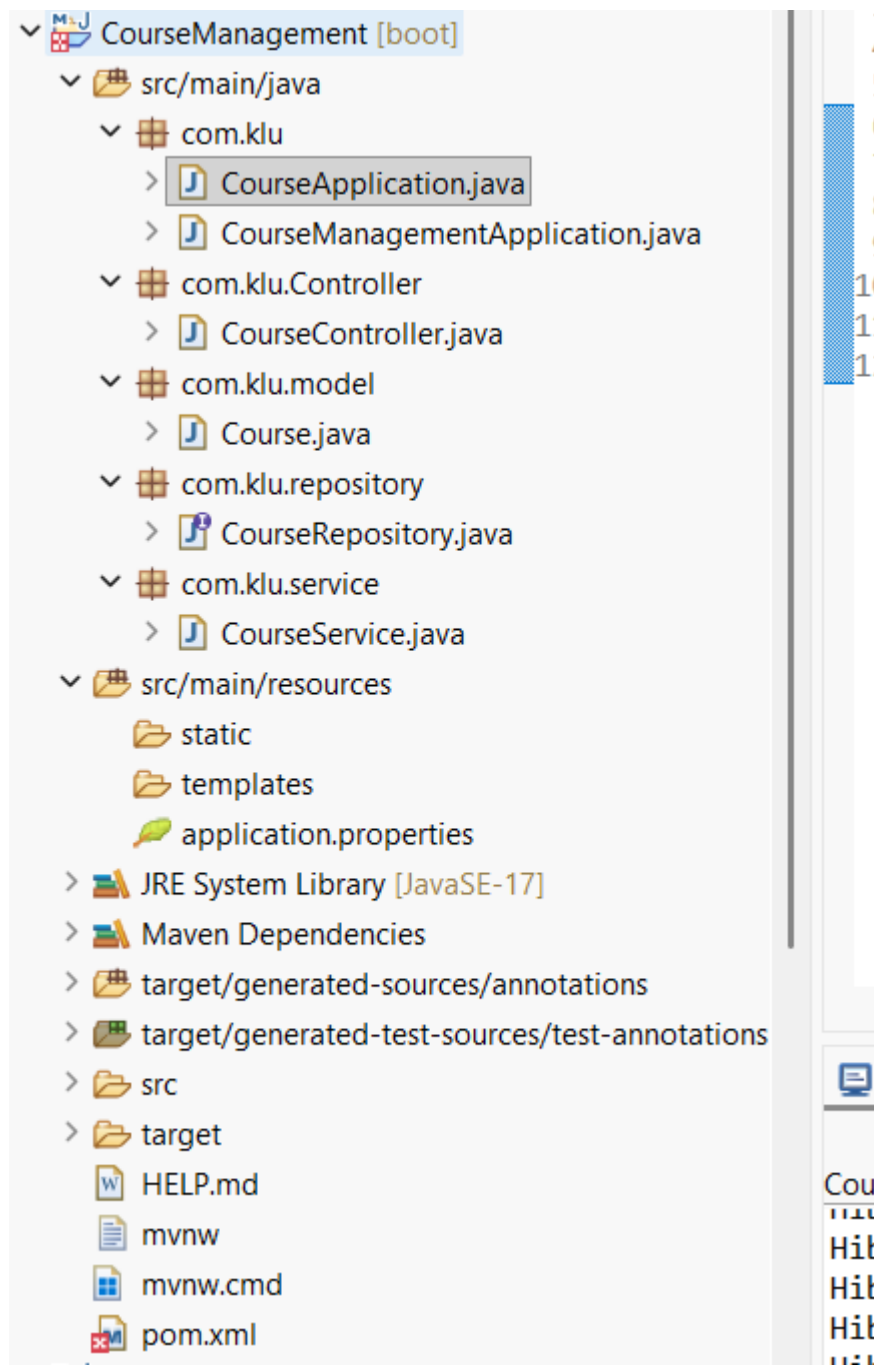
Select these dependencies:

- Spring Web
- Spring Data JPA
- MySQL Driver
- Lombok (optional)

Click Finish

Eclipse will create the project.

Create the following :



CourseApplication.java:

package com.klu;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class CourseApplication {

```
public static void main(String[] args) {  
    SpringApplication.run(CourseApplication.class, args);  
}  
}
```

CourseController.java:

```
package com.klu.Controller;  
  
import java.util.List;  
import java.util.Optional;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.http.HttpStatus;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;  
  
import com.klu.model.Course;  
import com.klu.service.CourseService;  
  
@RestController  
@RequestMapping("/courses")  
public class CourseController {  
  
    @Autowired  
    private CourseService service;  
  
    // Add Course  
    @PostMapping  
    public ResponseEntity<?> addCourse(@RequestBody Course course) {  
  
        Course savedCourse = service.addCourse(course);  
        return ResponseEntity.status(HttpStatus.CREATED).body(savedCourse);  
    }  
}
```

```
}
```

```
// Get All Courses
```

```
@GetMapping
```

```
public ResponseEntity<List<Course>> getAllCourses() {
```

```
    List<Course> courses = service.getAllCourses();
```

```
    return ResponseEntity.ok(courses);
```

```
}
```

```
// Get Course By ID
```

```
@GetMapping("/{id}")
```

```
public ResponseEntity<?> getCourseById(@PathVariable Long id) {
```

```
    Optional<Course> course = service.getCourseById(id);
```

```
    if (course.isPresent()) {
```

```
        return ResponseEntity.ok(course.get());
```

```
    } else {
```

```
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
```

```
            .body("Course not found");
```

```
    }
```

```
}
```

```
// Update Course
```

```
@PutMapping("/{id}")
```

```
public ResponseEntity<?> updateCourse(@PathVariable Long id, @RequestBody Course course) {
```

```
    try {
```

```
        Course updatedCourse = service.updateCourse(id, course);
```

```
        return ResponseEntity.ok(updatedCourse);
```

```
    } catch (Exception e) {  
        return ResponseEntity.status(HttpStatus.NOT_FOUND)  
            .body("Course not found for update");  
    }  
}
```

// Delete Course

```
@DeleteMapping("/{id}")  
public ResponseEntity<?> deleteCourse(@PathVariable Long id) {  
  
    try {  
        service.deleteCourse(id);  
        return ResponseEntity.ok("Course deleted successfully");  
  
    } catch (Exception e) {  
        return ResponseEntity.status(HttpStatus.NOT_FOUND)  
            .body("Course not found");  
    }  
}
```

// Search Course by Title

```
@GetMapping("/search/{title}")  
public ResponseEntity<?> searchCourse(@PathVariable String title) {  
  
    List<Course> courses = service.searchByTitle(title);  
  
    if (courses.isEmpty()) {  
        return ResponseEntity.status(HttpStatus.NOT_FOUND)  
            .body("No courses found");  
    }  
}
```

```
        return ResponseEntity.ok(courses);
    }
}
```

Course.java:

```
package com.klu.model;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
public class Course {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long courseId;
```

```
    private String title;
```

```
    private String duration;
```

```
    private double fee;
```

```
    public Course() {}
```

```
    // Getter and Setter for courseId
```

```
    public Long getCourseId() {
```

```
        return courseId;
```

```
    }
```

```
    public void setCourseId(Long courseId) {
```

```
        this.courseId = courseId;
```

```
    }
```

```
    // Getter and Setter for title
```

```
    public String getTitle() {
```

```

        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    // Getter and Setter for duration
    public String getDuration() {
        return duration;
    }

    public void setDuration(String duration) {
        this.duration = duration;
    }

    // Getter and Setter for fee
    public double getFee() {
        return fee;
    }

    public void setFee(double fee) {
        this.fee = fee;
    }
}

```

CourseRepository.java:

```

package com.klu.repository;

import com.klu.model.Course;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface CourseRepository extends JpaRepository<Course, Long> {

```



```
List<Course> findByTitleContainingIgnoreCase(String title);  
}
```

CourseService.java:

```
package com.klu.service;
```

```
import com.klu.model.Course;
```

```
import com.klu.repository.CourseRepository;
```

```
import org.springframework.stereotype.Service;
```

```
import java.util.List;
```

```
import java.util.Optional;
```

```
@Service
```

```
public class CourseService {
```

```
    private final CourseRepository repo;
```

```
    public CourseService(CourseRepository repo) {
```

```
        this.repo = repo;
```

```
    }
```

```
    public Course addCourse(Course course) {
```

```
        return repo.save(course);
```

```
    }
```

```
    public List<Course> getAllCourses() {
```

```
        return repo.findAll();
```

```
    }
```

```
    public Optional<Course> getCourseById(Long id) {
```

```
        return repo.findById(id);
```

```
    }
```

```
public Course updateCourse(Long id, Course courseDetails) {
```

```
    Course course = repo.findById(id).orElseThrow();
```

```
    course.setTitle(courseDetails.getTitle());
```

```
    course.setDuration(courseDetails.getDuration());
```

```
    course.setFee(courseDetails.getFee());
```

```
    return repo.save(course);
```

```
}
```

```
public void deleteCourse(Long id) {
```

```
    repo.deleteById(id);
```

```
}
```

```
public List<Course> searchByTitle(String title) {
```

```
    return repo.findByTitleContainingIgnoreCase(title);
```

```
}
```

```
}
```

application.properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/coursedb
```

```
spring.datasource.username=root
```

```
spring.datasource.password=YOUR Password
```

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

```
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
```

```
server.port=8080
```

=====

```
Pom.xml: <?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
    <relativePath/>
</parent>

<groupId>com.klu</groupId>
<artifactId>course</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>CourseManagement</name>
<description>Spring Boot Course CRUD Project</description>

<properties>
    <java.version>17</java.version>
</properties>

<dependencies>

    <!-- Spring Boot Web -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
```

<!-- Spring Boot JPA -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-data-jpa</artifactId>

</dependency>

<!-- MySQL Driver -->

<dependency>

<groupId>com.mysql</groupId>

<artifactId>mysql-connector-j</artifactId>

<scope>runtime</scope>

</dependency>

<!-- Lombok (Optional) -->

<dependency>

<groupId>org.projectlombok</groupId>

<artifactId>lombok</artifactId>

<optional>true</optional>

</dependency>

<!-- Testing -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-test</artifactId>

<scope>test</scope>

</dependency>

</dependencies>

<build>

<plugins>

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
</plugin>

</plugins>
</build>

</project>
```

```
CourseManagementApplication:
package com.klu;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CourseManagementApplication {

    public static void main(String[] args) {
        SpringApplication.run(CourseManagementApplication.class, args);
    }

}
```

After This

Run the project:

CourseApplication.java

Run As → Spring Boot App

Now Go To the My SQL workbench:

Step 1: Create Database

Open MySQL Workbench or MySQL terminal and run:

CREATE DATABASE coursedb;

Step 2: Run Application

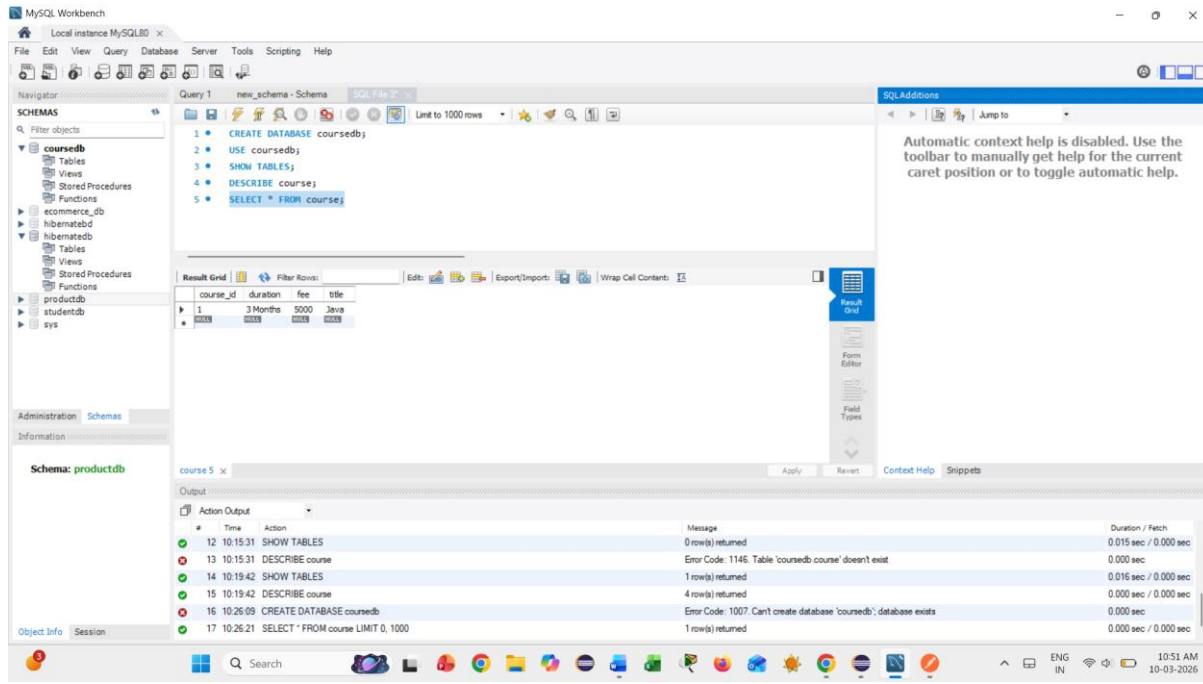
Run: CourseApplication.java

Step 3:

USE coursedb;

SHOW TABLES;

#Spring Boot creates the table automatically.



NOW IN THE POSTMAN APP:

1 Add Course (POST)

URL: <http://localhost:8080/courses>

Body → raw → JSON

```
{
  "title": "Java Programming",
  "duration": "3 months",
  "fee": 5000
}
```

Click Send;

Check MySQL:

SELECT * FROM course;

2 View All Courses(GET)

URL: <http://localhost:8080/courses>

Body → raw → JSON

```
[{  
  "courseId":1,  
  "title":"Java Programming",  
  "duration":"3 Months",  
  "fee":5000  
}]
```

3 Get Course by ID:(GET)

URL: <http://localhost:8080/courses/1>

Click **Send**;

If course exists: 200 OK

IF not : 404 NOTFound

Update Course(PUT):

URL: <http://localhost:8080/courses/1>

Body → raw → JSON

```
{  
  "title":"Advanced Java",  
  "duration":"4 Months",  
  "fee":7000  
}
```

Output:

200 OK

Delete Course(DELETE):

URL: <http://localhost:8080/courses/1>

Click send;

Output:

200 OK(Course deleted successfully)

Search Course by Title:(GET):

URL: GET <http://localhost:8080/courses/search/java>

Output:

```
[
{
  "courseId":1,
  "title":"Java Programming",
  "duration":"3 Months",
  "fee":5000
}]
```

Test Invalid Cases (Important for Assignment):

Example tests:

Test	Result
GET <code>/courses/100</code>	404 NOT_FOUND
DELETE <code>/courses/100</code>	NOT_FOUND
POST empty data	BAD_REQUEST

Example: GET ([URL:http://localhost:8080/courses/100](http://localhost:8080/courses/100))

NOTE: The thing we do in the postman will be reflected in MY SQL WORKBENCH to see this you have to run the highlight part.

Query 1

SQL File 3* x

S



Limit to 10

1 •

2 •

3 •

4 •

5 •

CREATE DATABASE coursedb;

USE coursedb;

SHOW TABLES;

DESCRIBE course;

SELECT * FROM course;

Execute the selected portion of the script or everything.

Result Grid

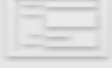


 Filter Rows:

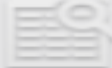
	course_id	duration	fee	title
▶	2	3 months	5000	Java Programming
•	NULL	NULL	NULL	NULL



Result Grid



Form Editor



Field Types



Result 1

Result 2

course 3 x

Apply

C